

An Efficient and Accurate Stochastic Number Generator Using Even-Distribution Coding

Aidyn Zhakatayev^{ID}, Kyoungsoon Kim, Kiyoun Choi, *Fellow, IEEE*, and Jongeun Lee^{ID}, *Member, IEEE*

Abstract—Stochastic computing (SC) is a promising approach for low-power and low-cost applications with the added benefit of high error tolerance. However, the high overhead of generating stochastic bitstreams can offset the advantages of SC especially when a large number of bitstreams are needed. In this paper, we propose a new stochastic number generator (SNG) that significantly reduces area and energy while improving accuracy. Experimental results show that the proposed SNG can reduce energy by more than 72% compared with the state-of-the-art designs.

Index Terms—Bit parallelism, bit shuffling, decorrelation, even-distribution code, linear feedback shift register (LFSR), stochastic computing.

I. INTRODUCTION

STOCHASTIC computing (SC) was first introduced in the 1960's when computing devices were not very reliable [2]. With the advent of solid-state devices and integrated circuits, the need for and interest in SC quickly dwindled but was rekindled recently due to the extreme miniaturization of transistor devices threatening the deterministic operation of computing devices, and the surge of computational complexity in applications such as deep neural networks.

SC [3] represents a number using a sequence of bits, which we call *bitstream* throughout this paper, where the frequency of 1s determines the value of the bitstream. Take for instance the bitstreams given to the AND gate as input in Fig. 1(a). The

Manuscript received June 28, 2017; revised October 6, 2017; accepted November 26, 2017. Date of publication January 4, 2018; date of current version November 20, 2018. This work was supported in part by the Nano-Material Technology Development Program through the National Research Foundation of Korea (NRF) through the Ministry of Science, ICT and Future Planning under Grant 2016M3A7B4909668, in part by the Basic Science Research Program through NRF funded by the Ministry of Education under Grant NRF-2017R1D1A1B03033591, in part by the KIST Institutional Program under Project 2E27330-17-P026, and in part by the Free Innovative Research Fund of UNIST under Grant 1.170067.01. A preliminary version of this paper appeared in [1]. This version extends it with more analysis, new techniques to reduce correlation, and more experimental results. This paper was recommended by Associate Editor S. Bhunia. (*Corresponding author: Jongeun Lee.*)

A. Zhakatayev and J. Lee are with the School of Electrical and Computer Engineering, Ulsan National Institute of Science and Technology, Ulsan 44919, South Korea (e-mail: jlee@unist.ac.kr).

K. Kim is with the DMC Research and Development Center, Samsung Electronics, Seoul 06765, South Korea.

K. Choi is with the Department of Electrical and Computer Engineering, Seoul National University, Seoul 08826, South Korea.

Color versions of one or more of the figures in this paper are available online at <http://ieeexplore.ieee.org>.

Digital Object Identifier 10.1109/TCAD.2018.2789732

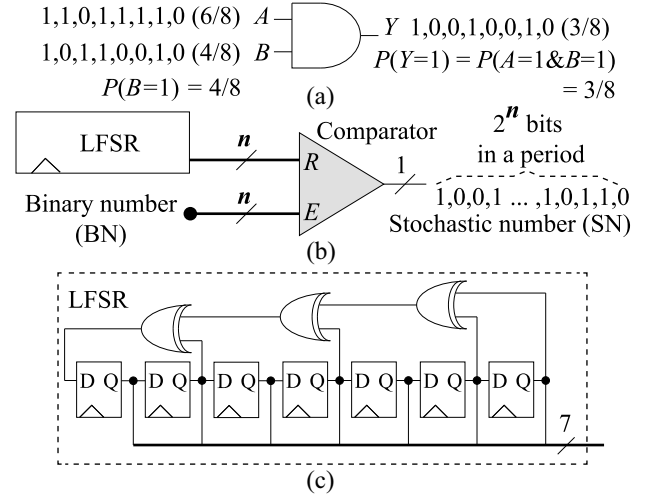


Fig. 1. Stochastic arithmetic operation and conventional SNG. (a) Multiplication of two SNs using an AND gate. (b) SNG consisting of an n -bit LFSR and an n -bit comparator. (c) Example of 7-bit LFSR.

bitstream for A has six 1s out of 8, which can be interpreted as meaning $6/8 = 0.75$, and similarly that of B can be interpreted as $4/8 = 0.5$. In this way, one can represent any value in the range of $[0, 1]$ using a bitstream, and the precision of the representation can be increased by using longer bitstreams.

Now the output Y of the AND gate is another bitstream, whose signal probability, or the probability of Y being 1, i.e., $P(Y) \equiv P(A \wedge B)$, equals $P(A)P(B)$ provided that the two input bitstreams are uncorrelated. Thus an AND gate can perform multiplication in SC. Though this way of computing may not produce an exact result, SC is less susceptible to random bit errors than conventional binary representation as every bit has an equal weight. Also arithmetic operations can be implemented with much less cost such as an AND gate for multiplication. This motivates the use of SC in many application where exact computation is not required, including neural networks [4], low-density parity-check [5], median filters [6], image processing [7], and other applications where certain errors on the operation level can be tolerated [8].

However, one of the largest disadvantages of SC is conversion overhead. Since most input is stored in the form of conventional binary representation, one needs a circuit that converts a binary number (BN) into a stochastic bitstream, which is called stochastic number generator (SNG). Fig. 1(b)

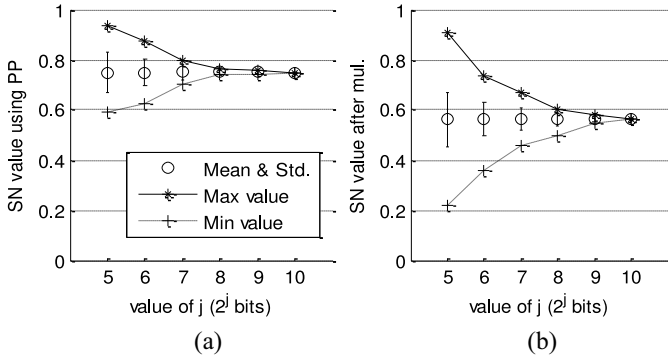


Fig. 2. SN generation for 0.75 with PP using an SNG with a 10-bit LFSR. (a) Distribution of SN values. (b) Multiplication of two SNs using an AND gate.

illustrates an SNG for n -bit BN, consisting of an n -bit linear feedback shift register (LFSR) and an n -bit comparator. The comparator generates 1 if the n -bit pseudo-random number generated by the LFSR is less than BN, and has the same period as LFSR, which is $2^n - 1$. Fig. 1(c) illustrates LFSR. Other random sources may be used but LFSR is most common due to its simplicity. Compared to SC logic itself, SNG adds a quite large overhead in terms of both area and power [9]–[11], and one often needs a number of SNGs, which can significantly offset the advantage of SC.

Though there has been some research into better random number generation methods in various domains [12]–[14], they mostly focus on improving the quality of randomness rather than reducing hardware cost. A recent work [11] proposes reducing hardware area by sharing an LFSR between two SNGs. However, simply sharing an LFSR among different SNGs can cause a significant accuracy degradation due to the increased correlation between stochastic bitstreams, and the degradation rises rapidly as an LFSR is shared by a greater number of SNGs.

In this paper, we propose an efficient way of generating high quality stochastic bitstreams. Our scheme is both more efficient and more accurate than the conventional LFSR-comparator-based scheme. It is many times more efficient than the conventional scheme as it requires asymptotically one store unit, i.e., D-F/F (D-type flip-flop), per stochastic bit. Our scheme is more accurate, since our SNG distributes the number of 1s evenly over the entire bitstream.

The rest of this paper is organized as follows. Section II explains the proposed SNG, Section III presents the methods to reduce correlation in the proposed SNG, Section IV shows our experimental results, and Section V concludes this paper.

II. PROPOSED STOCHASTIC NUMBER GENERATOR

A. Problems With Conventional SNG

The conventional SNG consisting of an LFSR and a comparator as illustrated in Fig. 1(b) has two shortcomings. First even though the SNG circuit contains an n -bit LFSR, which is capable of generating n random bits at a time, the SNG can only generate a single stochastic bit at a time. If all the n random bits of the LFSR can be utilized, the SNG needs to be

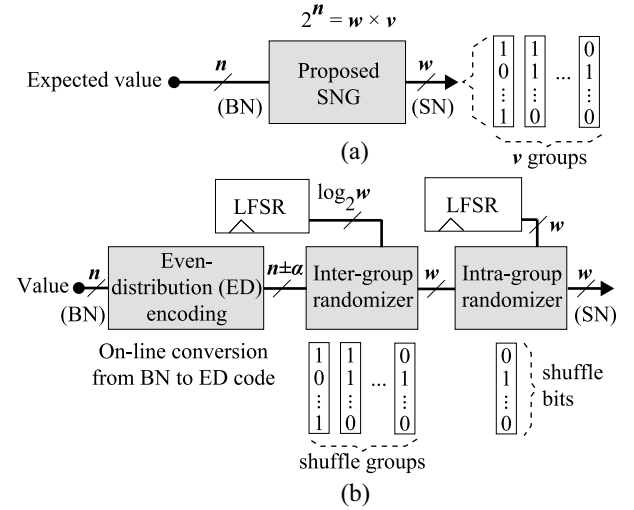


Fig. 3. Overview of the proposed SNG. (a) SNG with BN input and SN output, where v groups are generated and each group has w bit-width. (b) Proposed SNG consists of three parts: 1) ED encoding; 2) intergroup; and 3) intragroup randomizer.

activated n times less, greatly reducing the energy consumption due to SNG.

Second, while progressive precision (PP) [15] or early decision termination [16] is an important advantage of SC, the stochastic bitstream generated by the conventional SNG can be very inaccurate especially in the early phase. Consider an SNG with a 10-bit LFSR and a certain target BN. The value of the generated bitstream (computed as the average of the bit values) after seeing all the $N = 2^{10} - 1$ bits always converges to the target value, since N is the period of the LFSR. However, a substring of the bitstream, as is used in the PP scenario, does not have to, and may have a very different statistics from that of the original full-length version. Fig. 2(a) shows the statistics for shorter versions of the bitstream, where the LFSR is 10-bit and the target BN is 0.75. For the full-length version, the value of the bitstream always matches 0.75, but it deviates significantly for shorter versions. Fig. 2(b) shows the distribution of stochastic number (SN) value after multiplication of two SNs using an AND gate; the error is very large especially for shorter versions of the full bitstream.

B. Overview of the Proposed SNG

To overcome the above mentioned limitations, we propose a new SNG method based on *shuffling* and *partitioning*. Instead of generating one bit at a time, our proposed SNG *shuffles* an existing bitstream, thus generating multiple bits at once. We now need to solve two problems: 1) how to provide an initial bitstream and 2) how to do the shuffling efficiently. The second problem is addressed by partitioning the bitstream into multiple groups and do the shuffling at the group level. This partitioning also helps maintain higher accuracy in the PP scenario. Finally, generating an initial bitstream can be done easily, since we know the target BN. Based on the target BN we can determine the number of 1s in the bitstream; the position of 1s and 0s in the initial bitstream is irrelevant as we shuffle them away.

(a) $L=76_{\text{dec}}$ ED code: (1010,2,2)

Sat. bit	Row index	Group ID							Weight (b=3)
		0	1	2	3	4	5	6	
s[3]	1	1	1	1	1	1	1	1	24
s[2]	1	1	1	1	1	1	1	1	12
s[1]	0	1	1	1	1	1	1	1	6
s[0]	1	1	1	1	1	1	1	1	3

(b) $L=76_{\text{dec}}$ ED code: (1010,1,1)

Sat. bit	Row index	Group ID							Weight b=1
		0	1	2	3	4	...	14	
0	3	1	1	1	1	1	...	1	8
1	2	1	1	1	1	1	...	1	4
0	1	1	1	1	1	1	...	1	2
1	0	1	1	1	1	1	...	1	1

Fig. 4. ED encoding, where white space means zero. ED code for (a) 237_{dec} with seven groups (#columns) and 4 bits per group (#rows) when $b = 3$ and (b) 76_{dec} with 15 groups and 4 bits when $b = 1$.

Fig. 3 illustrates the overall flow of our proposed SNG. Our SNG consists of three parts as shown in Fig. 3(b): 1) even-distribution (ED) encoding; 2) intergroup randomizer; and 3) intragroup randomizer. ED encoding makes 1s evenly distributed among groups; intergroup randomizer shuffles the order of groups; and intragroup randomizer shuffles bits within each group. All steps can be implemented by combinational logic except for two LFSRs, which makes the proposed SNG very efficient in terms of area and power.

C. Even-Distribution Encoding

The objective of ED encoding is to construct v groups, where the difference in the number of 1s between groups is at most b (b is a design parameter). In other words, $\max_i g_i - \min_i g_i \leq b$, where g_i is the number of 1s for the i th group.

To achieve ED encoding and to facilitate efficient design of a bit-parallel SNG as illustrated in Fig. 3(a), we propose a tabular encoding scheme. Fig. 4 shows two examples of our ED encoding for $b = 3$ and $b = 1$. In the figure, each column represents a group and each row represents a bit in a group (empty cells or cells occupied by α mean zero in the figure). These bits are not the bits that are ultimately generated by our SNG, but merely represents the value or quantity carried by each group in a positional number system similar to the conventional binary representation. But unlike conventional binary, each bit has a weight of $2^i b$, where i is the zero-based row index. Moreover there is a hidden bit 1 for every group if the zeroth saturation bit, shown on the left side of the table, is 1. Thus the value of a group, whose bit vector is $\vec{a}_i$, is

$$g_i = \sum_{j=0}^{k-1} a_i[j] \cdot 2^j \cdot b + s[0] \cdot b \quad (1)$$

where k is the number of rows and $s[0]$ is the zeroth saturation bit. For example, group 1 in Fig. 4(a) has 1100_{bin} , whose value is $39 (= 1 \cdot 24 + 1 \cdot 12 + 0 \cdot 6 + 0 \cdot 3 + 1 \cdot 3)$ and group 3 has 1011_{bin} , whose value is $36 (= 1 \cdot 24 + 0 \cdot 12 + 1 \cdot 6 + 1 \cdot 3 + 1 \cdot 3)$. Their difference is 3, which is bounded by b .

(a) $L = 414$

Sat. bit	Row index	Group ID						Weight
		0	1	2	...	30	31	
0	4							16
1	3	1	1	1	1	1	1	8
0	2	1	1	1	1	1	α	4
0	1						1	2
1	0						1	1

(a) $L = 415$

Sat. bit	Row index	Group ID						Weight
		0	1	2	...	30	31	
0	4							16
1	3	1	1	1	1	1	1	8
0	2	1	1	1	1	1	α	4
0	1						1	2
1	0						1	1

(a)

(a) $L = 159$

Sat. bit	Row index	Group ID						Weight
		0	1	2	...	30	31	
0	4							16
0	3							8
0	2	1	1	1	1	1	α	4
0	1						1	2
1	0						1	1

(b) $L = 160$

Sat. bit	Row index	Group ID						Weight
		0	1	2	...	30	31	
0	4							16
0	3							8
1	2	1	1	1	1	1	1	4
0	1							2
1	0	α						1

(b)

(b) $L = 127$

Sat. bit	Row index	Group ID						Weight
		0	1	2	...	30	31	
0	4							16
0	3							8
0	2							4
1	1	1	1	1	1	1	1	2
1	0	1	1	1	1	1	α	1

(c) $L = 128$

Sat. bit	Row index	Group ID						Weight
		0	1	2	...	30	31	
0	4							16
0	3							8
0	2	α						4
0	1	1	1	1	1	1	1	2
1	0	1	1	1	1	1	1	1

(c)

Fig. 5. Operations on ED code ($b = 1$). (a) Shift: $(s, 2, 30) \rightarrow (s, 2, 31)$. (b) Return: $(s, 2, 31) \rightarrow (s', 0, 0)$. (c) Carry: $(s, 0, 31) \rightarrow (s', 2, 0)$.

The total value of an ED code is simply $\sum_{i=0}^{v-1} g_i$, which should (closely) match the target binary value in unipolar coding. The error due to our ED encoding is at most $\lfloor b/2 \rfloor$, which equals zero if $b = 1$. (Thus b represents the granularity of ED encoding.) Using larger b reduces the number of ED code bits, achieving higher efficiency in terms of storage and logic area while sacrificing accuracy.

Finding the ED code of a given BN L boils down to finding the pivot position α , satisfying the following rules (see Fig. 4).

- 1) Each of the upper rows of α should be all zeros or all ones (called *saturation bits*).
- 2) The left side of α should be all ones whereas the right side of α , including α , should be all zeros.
- 3) The lower left side should be all zeros, whereas the remaining lower space should be all ones.

This guarantees that the difference in value among groups is limited to b . The maximum difference occurs between the pivot group and the group to its left, and is $2^y b - \sum_{i=0}^{y-1} 2^i b = b$, where y is the row index of α (we can ignore the rows above α because they are identical in all groups).

We represent ED code as a triplet (s, y, x) , where s is a bit vector of length k , $s \equiv s[k-1]s[k-2] \dots s[0]$, representing

saturation bits, y is the row index ($0 \leq y < k$), and x is the group ID ($0 \leq x < v$).

The value L of an ED code can be computed by considering the following factors. Since the effect of b is merely to multiply the value of an ED code by the same factor [see (1)], in the following we drop b for brevity.

- 1) *Saturation Bits*: $\sum_{i=y+1}^{k-1} s[i] \cdot 2^i \cdot v$.
- 2) *The Pivot's Left Groups*: $2^y \cdot x$.
- 3) *The Pivot and its Right Groups*: $\sum_{i=0}^{y-1} 2^i \cdot (v - x)$.
- 4) *The Zeroth Saturation Bit*:¹ $s[0] \cdot v$

Since we do not actually use saturation bits $s[i]$ for $0 < i \leq y$, we define them to be zero. Then we have

$$L = \sum_{i=0}^{k-1} s[i] \cdot 2^i \cdot v + 2^y \cdot x + \sum_{i=0}^{y-1} 2^i \cdot (v - x). \quad (2)$$

To see that our ED coding as defined in (2) can represent any BN without error when $b = 1$, let us introduce the following operations. See Fig. 5 for illustrations of the operations.

- 1) *Shift* $\equiv (s, y, x) \rightarrow (s, y, x + 1)$, where $x < v - 1$.
- 2) *Return* $\equiv (s, y, v - 1) \rightarrow (s', 0, 0)$, where $y > 0$ and

$$s'[i] = \begin{cases} s[i], & \text{if } i \neq y \\ 1, & \text{if } i = y. \end{cases}$$

- 3) *Carry* $\equiv (s, 0, v - 1) \rightarrow (s', y', 0)$, where $y' = \min\{i \mid s[i] = 0\}$ and

$$s'[i] = \begin{cases} s[i], & \text{if } i > y' \\ 0, & \text{if } 0 < i \leq y' \\ 1, & \text{if } i = 0. \end{cases}$$

Lemma 1: Shift, return, and carry operations as defined above increase the value of ED code by 1.

Proof: The correctness for shift operation follows from (2):

$$\begin{aligned} (s, y, x + 1) &= \sum_{i=0}^{k-1} s[i] 2^i v + 2^y (x + 1) + \sum_{i=0}^{y-1} 2^i (v - (x + 1)) \\ &= \sum_{i=0}^{k-1} s[i] 2^i v + 2^y x + \sum_{i=0}^{y-1} 2^i (v - x) + 2^y - \sum_{i=0}^{y-1} 2^i \\ &= (s, y, x) + 1. \end{aligned}$$

For return operation, we know $s[y] = 0$ when $y > 0$, from the fact that $(s, y, v - 1)$ is a valid ED code. Thus,

$$\begin{aligned} (s', 0, 0) &= \sum_{i=0}^{k-1} s'[i] 2^i v = \sum_{i=0}^{k-1} s[i] 2^i v + 2^y v \\ &= \sum_{i=0}^{k-1} s[i] 2^i v + 2^y v - 2^y + \sum_{i=0}^{y-1} 2^i + 1 \\ &= \sum_{i=0}^{k-1} s[i] 2^i v + 2^y (v - 1) + \sum_{i=0}^{y-1} 2^i (v - (v - 1)) + 1 \\ &= (s, y, v - 1) + 1. \end{aligned}$$

For carry operation, since $y' = \min\{i \mid s[i] = 0\}$, we have

$$s'[i] = \begin{cases} 0, & \text{for } i = y' \\ 1, & \text{for } i < y'. \end{cases}$$

¹Due to our construction, $s[0]$ can be 0 only if $s[i] = 0$ for $\forall i > 0$.

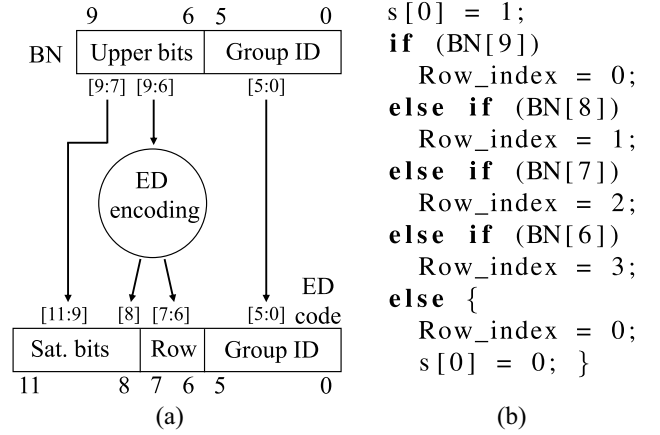


Fig. 6. ED encoding algorithm to convert 10-bit BN into ED code, consisting of 64 groups with 16 bits in each group. (a) Overview. (b) Algorithm.

Therefore,

$$\begin{aligned} (s, 0, v - 1) &= \sum_{i=0}^{k-1} s[i] 2^i v + v - 1 \\ &= \sum_{i=y'+1}^{k-1} s[i] 2^i v + \sum_{i=0}^{y'-1} 2^i v + v - 1 \\ (s', y', 0) &= \sum_{i=0}^{k-1} s'[i] 2^i v + \sum_{i=0}^{y'-1} 2^i v \\ &= \sum_{i=y'+1}^{k-1} s[i] 2^i v + 1 \cdot 2^0 v + \sum_{i=0}^{y'-1} 2^i v \\ &= (s, 0, v - 1) + 1. \end{aligned}$$

The minimum value in our ED code is zero, represented by $(\vec{0}, 0, 0)$, where $\vec{0}$ is a zero vector. The maximum value is $v \cdot 2^k - 1$, represented by $(\vec{1}, 0, v - 1)$, where $\vec{1}$ is a one vector. Since the “from” part of the three operations covers the entire range between the minimum and maximum values (excluding the maximum value), it is clear that one can traverse from zero to the maximum value within our ED code space. Therefore, our ED coding can indeed represent any BN without error.

The BN to ED encoder can be easily implemented in combinational logic. Fig. 6 illustrates an example circuit where BN is 10-bit and ED code has 64 groups with 16 bits in each group and the weight parameter of 1. There are four rows in this case, so the ED code for this example consists of 6 bits to identify one of 64 groups, 2 bits to identify one of four rows, and 4 saturation bits. As shown in Fig. 6(a), the Group ID comes directly from the lowest 6 bits of BN. The upper bits of BN determines the saturation bits (Sat. bits in the figure), except for the last bit ($s[0]$ in the algorithm), which is 1 if $BN \geq b \cdot v$, and 0 otherwise. The row index is easily calculated from the upper bits of BN as shown in Fig. 6(b).

D. Intergroup Randomization

The second step shuffles the order of groups so as to increase randomness. Let us call the groups with larger value

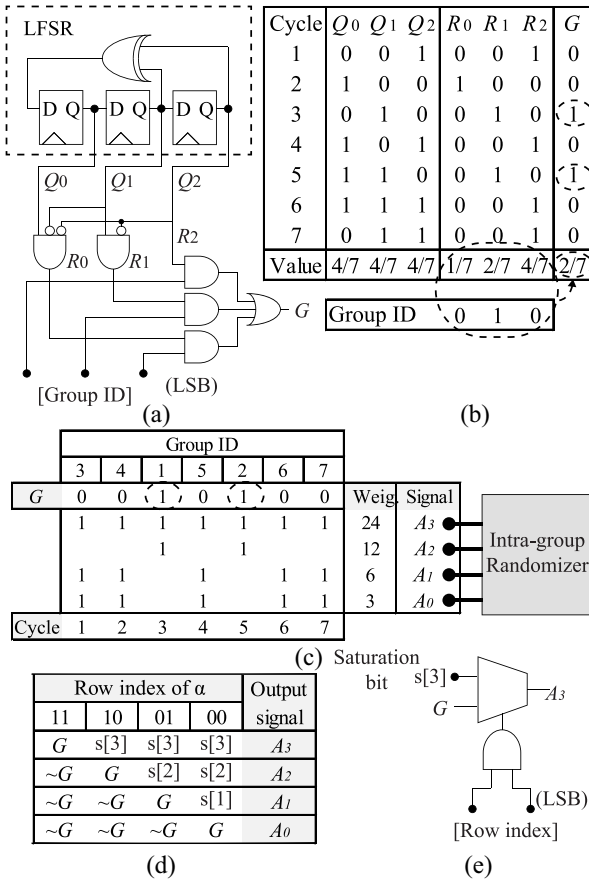


Fig. 7. Intergroup randomization, where the group index is from Fig. 4(a). (a) Group shuffling circuit. (b) Outputs of LFSR and of the group shuffling circuit. (c) Reordering groups according to G . (d) Generating output signals to be used as input for the next step, where $\sim G$ means logical negation of G . (e) Example logic for output signal A_3 .

BGs, which are the groups to the left of the pivot group. The objective of intergroup randomization is to shuffle the BGs with the other groups, for which we adopt the technique in [17]. Fig. 7(a) shows a circuit of intergroup randomization for the example of Fig. 4(a), where LFSR is used as the random source. In the example, the mean values of the bitstreams at R_2 , R_1 , and R_0 are 4:2:1, i.e., $\text{Prob}(R_i) \propto 2^i$, while the period of the LFSR is as close to v as possible. Now by taking the sum-of-products between the R_i signals and the binary representation of x (the group ID of α), we can get a new signal G , which has exactly x ones per period, but spread out as shown in Fig. 7(b). Then we only need to place a BG when $G = 1$ and one of the other groups when $G = 0$ as illustrated in Fig. 7(c).

Fig. 7(d) shows how to efficiently generate the output signals A_i ($0 \leq i < k$) that are passed to the next step, viz., the bit shuffling step. A key observation here is that the bit patterns of the groups are determined only by y , the row index of α , as follows:

$$A_i = \begin{cases} s[i], & \text{if } i > y \\ G, & \text{if } i = y \\ \neg G, & \text{if } i < y. \end{cases} \quad (3)$$

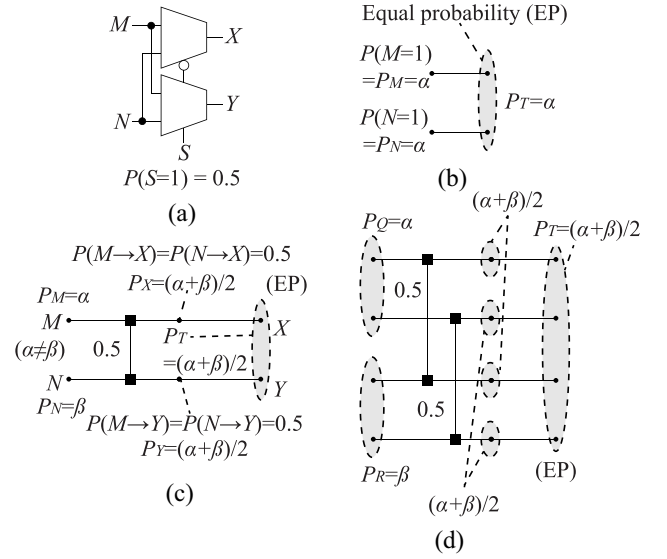


Fig. 8. Randomization network. (a) Swapper. (b) Definition of EP set. (c) Swapper symbol. (d) Constructing an EP set from two different EP sets.

For example, if $y = 0$, output A_i ($i > 0$) follows saturation bit $s[i]$, and A_0 agrees with G .

The logic in Fig. 7(d) can be easily implemented by a multiplexer with row bits as control input. Fig. 7(e), for instance, shows the logic for signal A_3 . The entire circuit for intergroup randomization can be implemented in combinational logic plus one LFSR.

E. Intragroup Randomization

To explain how bits are shuffled in the intragroup randomization step, let us first define the following terms. An equal probability (EP) set is a set of signals that have the same signal probability. A *swapper* consists of two muxes sharing two input signals as illustrated in Fig. 8(a) with complementary controls of probability 0.5. A swapper averages out input signal probabilities, producing an EP set. A swapper is denoted by the symbol in Fig. 8(c).

Two EP sets with f signals each can be merged into one EP set with $2f$ signals by using f swappers, where each swapper takes two input signals, one from each input EP set, as illustrated in Fig. 8(d). By applying this recursively, one can construct a randomization network that produces averaged-out versions of the input signals, if the number of input signals is a power of 2.

In our case, the outputs from the previous step have different weights, with A_i having twice the weight of A_{i-1} , and A_0 having the same weight as $s[0]$. Thus we duplicate the input signals A_i ($0 \leq i < k$) and $s[0]$ according to their weights, resulting in $2^k b$ bit lines in total as illustrated in Fig. 9. Moreover the bit lines originating from the same input signal are already an EP set, which allows for an efficient randomization network as illustrated in the figure. By exploiting the fact that some of the bit lines are already an EP set, we can reduce the number of swappers from $2^{k-1}b \cdot (k + \lceil \log b \rceil)$ to $(2^k - 1)b$ for $2^k b$ bit lines, achieving about $k/2$ times improvement. Note that this reduction affects not only cost but also

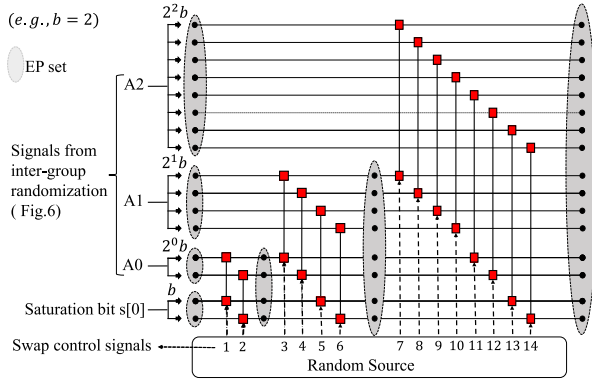


Fig. 9. Logic for intragroup randomization consisting of randomizing network and random sources. The input signals are from intergroup randomization.

the critical path delay of the circuit and the number of random sources required.

III. REDUCING CORRELATION AMONG RANDOM SOURCES

While the technique presented so far can provide high quality SNs in a cost effective way, a naïve implementation can produce a poor result for SC multiplication. In a simple implementation using a Fibonacci LFSR and feeding the bits of a Fibonacci LFSR to the bit shuffling circuit in order, we find the accuracy of SC multiplication can be very (see Section IV-F) frustrating. The main problem is the high correlation between the two operands of an SC multiplication, which is in large part determined by the quality of random sources and how the bits from the random sources are used in the bit shuffling circuit. In this section, we discuss a set of low-cost methods to make our SNG more robust for SC multiplication, which is a very common operation in many applications.

A. Observations

1) *Importance of Removing Temporal Correlation:* For our bit shuffling circuit generating $2^k b$ SN bits in each cycle, we need $(2^k - 1)b$ swappers and the same number of random bit sources. While this can be easily implemented by an LFSR of $(2^k - 1)b$ bits, the high temporal correlation of an LFSR can negatively impact the quality of the SN bitstream. The reason is the following. In each cycle the bit shuffling circuit processes different groups. But there are only one or two unique groups as far as the input to the bit shuffling circuit is concerned, and the v groups are merely duplicates of the few unique groups. Thus the bit shuffling circuit will see the same input in successive cycles with a high probability, and if the current input is not the same as the previous input, the next must be the same as either the current or the previous.

There are two types of LFSR, and Galois LFSR appears to have lower temporal correlation than Fibonacci LFSR, as demonstrated in Table I, though both have low quality compared with, e.g., GNU C library's `rand()` function.

2) *Different Importance Among Swap Control Signals:* The randomization network used for bit shuffling in Fig. 9 is highly asymmetric. Swapper 1, for instance, is connected to far more

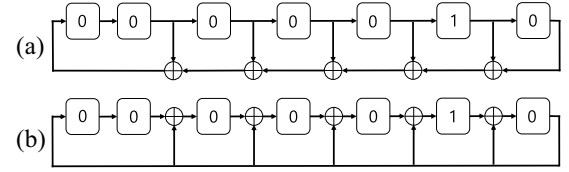


Fig. 10. Different LFSR implementations. (a) Fibonacci type and (b) Galois type. Both have the same period.

TABLE I
COMPARISON OF FIBONACCI AND GALOIS LFSRS

Cycle	Fibonacci LFSR	Galois LFSR
1	0000010	0000010
2	1000001	0000001
3	1100000	1011111
4	1110000	1110000

bit lines than the last swapper, and potentially has a higher impact. In fact the randomization network consists of k stages, and swap control signals of an earlier stage can affect more bit lines than those of a later stage, while there is no difference within the same stage.

3) *Dependence Among Swap Control Signals:* Swap control signals should be independent of each other for high quality bit shuffling. For example, in Fig. 9 if the first and third swap control signals always agree, this network can never move bit 1 or bit 7 to bit 3 (index is zero-based and starts from the bottom), reducing the randomness of shuffling. In general, swappers sharing the same bit line should be independent of each other in order to get good quality bit shuffling.

B. Design Exploration

Based on the observations we introduce three techniques that can reduce correlation even when the quality of random source is poor. While the design space is huge, our goal is *not* to explore it all but to find a simple enough design that can perform well in terms of multiplication accuracy.

1) *LFSR Design:* There are three factors that may affect the quality of an LFSR of a given bit-width: LFSR type, the number of taps, and the taps position. Fig. 10 illustrates two ways to implement an LFSR, commonly referred to as the Fibonacci and Galois type. In the Fibonacci type all bits except for the first one are shifted from the next bits and the first bit is computed from a XOR of possibly many bits, whereas in Galois type many internal bits can be updated simultaneously by two-input XORs. Thus we use the Galois type as it can change more bits simultaneously in a more unpredictable way than the Fibonacci type. For the same reason we can expect an LFSR with more taps to be less predictable, or suffer less from temporal correlation. Finally, for taps positions we explore *maximal LFSR functions*, which are tap positions generating all possible $2^n - 1$ states, where n is the number of bits.

2) *Mapping Between LFSR Bits and Swappers:* Even if we choose a maximal LFSR function with the maximum number of taps, taps are a limited resource, and not all swappers may be connected to the bit right after a tap. However, taking advantage of the fact that not all swap control signals are equally important (see

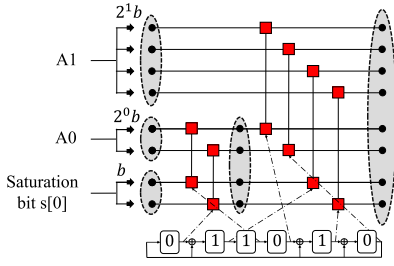


Fig. 11. Out-of-order mapping between LFSR bits and swappers.

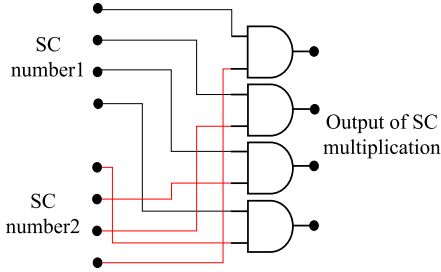


Fig. 12. Multiplication of two SC bitstreams in reverse order.

Section III-A2), we can assign LFSR bits immediately following an XOR gate to more important, or earlier-stage, swappers. This breaks the in-order mapping convention between LFSR bits and swappers, but in a disciplined way.

We can further explore all out-of-order mappings between LFSR bits and swappers, as illustrated in Fig. 11. Since the number of combinations to explore is huge—there are $(n!)^2$ different combinations, where n is the size of an LFSR and square is due to two operands—we randomly sample a large number of combinations to see the impact of out-of-order mapping.

3) *Bit Matching Order Between Multiplication Operands:* Bit shuffling, which is the last step of our SNG, generates $2^k b$ bits in parallel. As such, our SNG can be readily used in a bit-parallel SC circuit. More importantly, it gives us another exploration dimension, that is, bit matching order between two operands of a bit-parallel SC multiplication. If bit shuffling is perfect, no bit matching order would be significantly better or useful at all. But as it is not the case, out-of-order bit matching in SC multiplication, such as the reverse order matching illustrated in Fig. 12, may help compensate for some correlation among bits. There are $n!$ different combinations to explore, where n is the number of bits in a group ($= 2^k b$).

IV. EXPERIMENTS

We compare three cases: the conventional SNG (referred to as conventional) as shown in Fig. 7(b), the previous work [11] with shared LFSR (referred to as shared), and the proposed SNG. Unless noted otherwise, the length of LFSRs in the conventional and the shared case is 10. In the proposed SNG, the size of a group is 32 bits.

We evaluate both *accuracy* and *efficiency*. As we generate many stochastic bitstreams, their “values” as computed by converting them into equivalent BNs may not always agree. In fact it is common for these binary values to have variation.

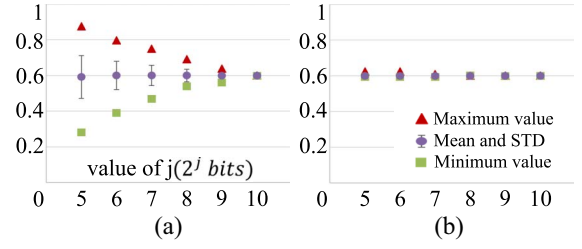


Fig. 13. Accuracy of SNG bitstreams, for the binary value of 0.6. The value j on x -axis means the bitstream is 2^j -bit long. (a) Conv. SNG. (b) Proposed SNG.

We use the statistics of these binary values such as mean and standard deviation (STD) to capture how closely these binary values agree with the target value. Further we say that one set of stochastic bitstreams is more *accurate* than another if its STD value (after converting into binary) is lower and both have the same mean value as the target value.

A. Accuracy of SNG Bitstream

We first evaluate the accuracy of SNG-generated bitstreams. Fig. 13(a) shows the statistics (mean, STD, and min/max values) of bitstreams generated for the target binary value of 0.6 in unipolar encoding using the conventional SNG and the proposed SNG. The accuracy of the shared case is the same as that of the conventional case because it simply shares an LFSR between multiple SNGs. In the graph the x -axis represents the length of bitstream in a logarithmic scale.

The graphs suggest that in both cases the mean is the same as the input binary value (thus unbiased), but the proposed SNG generates bitstreams that are much more accurate (i.e., lower STD) than the conventional case. Especially when the bitstream length is short, e.g., when using 2^5 -bit subsequences out of the total 2^{10} -bit stream as in PP [15], the proposed SNG's bitstream can be much more accurate than that of the conventional case.

B. Area, Delay, Power, and Energy

We have also implemented the three cases in Verilog RTL, and the designs are synthesized using TSMC 45-nm technology library with Synopsys Design Compiler. All SNGs are implemented in bit-parallel manner for fair comparison, generating 2^{10} bits every clock cycle. Fig. 14 summarizes the result. Compared with the conventional SNG, the proposed SNG reduces area, power, and energy by 85.7%, 88.6%, and 86.3%, respectively. Compared to the shared case, the reductions by the proposed SNG is 78.9%, 74.4%, and 72.1%, respectively. However, the critical path delay of the proposed SNG increases by 20% and 9% compared with the conventional and shared cases, respectively.

C. Effect of Random Source on SC Multiplication Accuracy

While the proposed SNG is very accurate in terms of the generated bitstream itself, the bitstreams may exhibit higher correlation than the conventional case [1], leading to lower accuracy when applied to certain operations such

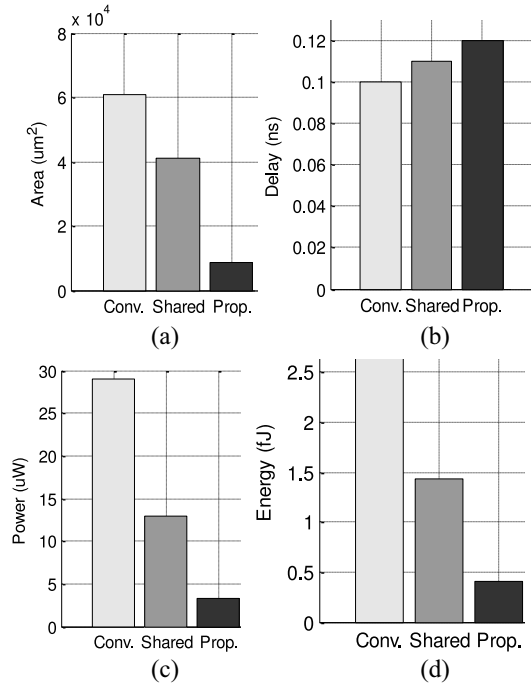


Fig. 14. Comparison of area, critical path delay, power, and energy to generate a 2^{10} -bit stream for the conventional, shared, and proposed cases. The length of LFSRs in the two previous approaches is 10 while the group size of the proposed SNG is 32 bits. (a) Area. (b) Critical path delay. (c) Power. (d) Energy.

as SC multiplication. Thus we evaluate the effectiveness of our enhancements introduced in Section III in improving the accuracy of SC multiplication result.

First, we explore different LFSR types and tap selections. As before, the target binary value for each SNG is 0.6 in unipolar encoding (there are two SNGs). We set the bitstream length to 2^{10} . Other settings are the same as before.

Fig. 15(a) shows the STD distribution of SC multiplication results when using various six-tap Fibonacci LFSRs as the random source. The x -axis shows STD ranges and the y -axis is the relative frequency, with the area under the graph being equal to 1. The variance in the STD is due to different tap selections, and the graph is generated by exploring all maximal six-tap Fibonacci LFSRs.

By comparison, replacing the random source with four- or six-tap Galois LFSRs reduces the STD distribution as shown in Fig. 15(b) and (c). Also we can see that the effect of LFSR type (Galois versus Fibonacci) is more significant than that of the number of taps, although both need to be optimized to obtain the best result.

These graphs further suggest that the tap selection can make a significant effect on the STD value, especially when using Galois LFSRs. For example, when using six-tap Galois LFSRs [see Fig. 15(c)], the difference in STD value when using the best as opposed to the worst LFSRs can be as high as $2\times$.

D. Effect of Out-of-Order Mapping and Bit Matching Order

While Galois LFSR improves the accuracy of SC multiplication considerably compared to using Fibonacci LFSR, to further improve the accuracy of the proposed SNG

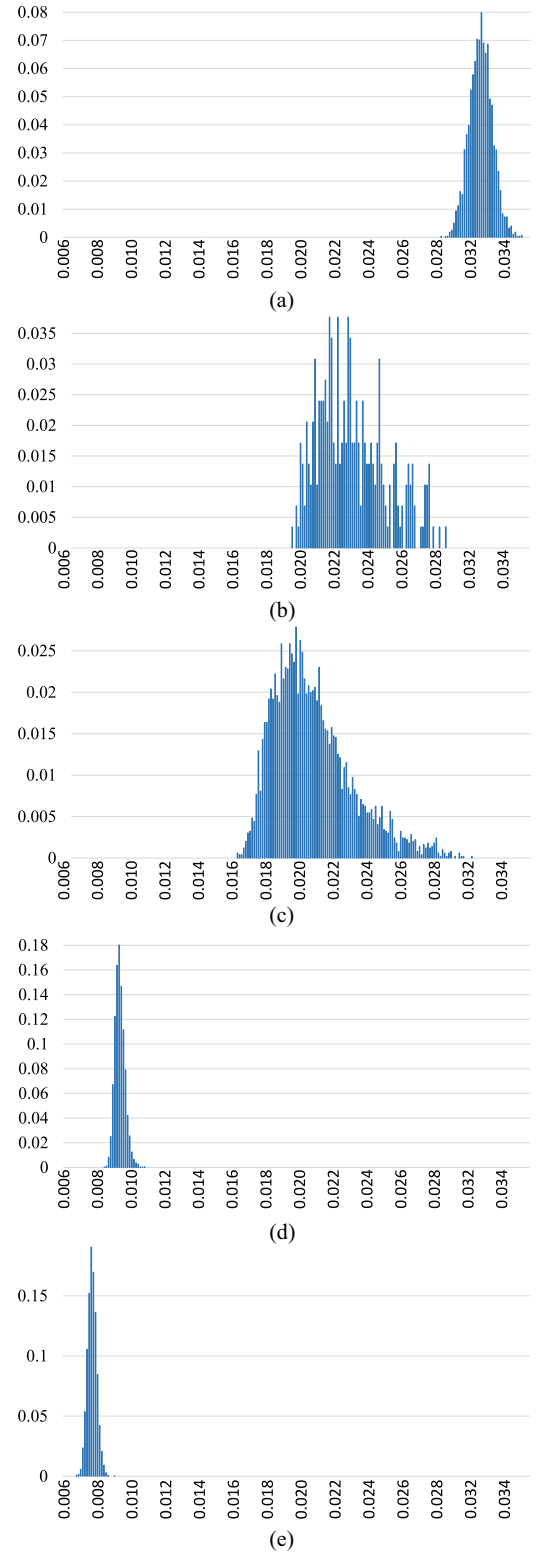


Fig. 15. Distribution of SC multiplication accuracy for different random sources. Accuracy is shown on the x -axis as measured in STD. The y -axis is relative frequency. (a) Using six-tap Fibonacci LFSRs as random source. (b) Using four-tap Galois LFSRs as random source. (c) Using six-tap Galois LFSRs as random source. Exploration of (d) mapping order between LFSR bits and swappers and (e) bit matching order between multiplication operands.

we explore the effect of optimizations. The optimizations are: 1) out-of-order mapping between LFSR bits and swappers and 2) bit matching order between multiplication operands. The

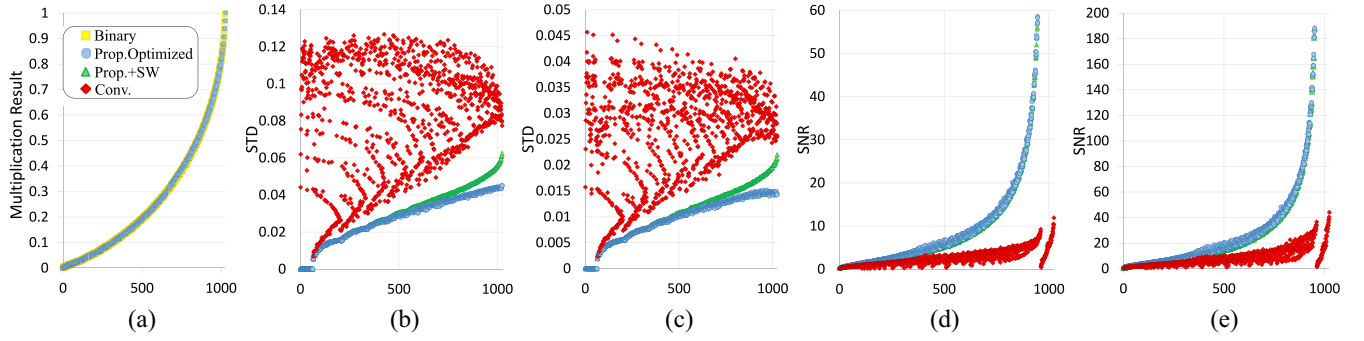


Fig. 16. SC multiplication accuracy for different input values. The x-axis shows different input value combinations, which are sorted in (a) by the binary multiplication result; in (b)–(e), they are sorted such that the Prop.+SW case is increasing. (a) Mean (2^5 -bit). (b) STD (2^5 -bit). (c) STD (2^8 -bit). (d) SNR (2^5 -bit). (e) SNR (2^8 -bit).

first optimization is applied on top of the best 6-tap Galois LFSR implementation, and its result is shown in Fig. 15(d). Likewise the second optimization is applied on top of the best result of the first optimization, and its result is shown in Fig. 15(e). For these two optimizations we explore only 5000 combinations each, which are generated randomly, since the total number of combinations is prohibitively large.

Fig. 15(d) suggests that changing the order between LFSR bits and swappers is very effective in improving accuracy. In fact, even though the result in Fig. 15(d) is based on the best one in Fig. 15(c), almost none of the 5000 random orderings generates a result close to Fig. 15(c). The graph suggests that using a random order between LFSR bits and swappers can improve the accuracy as much as $2\times$ compared with using in-order mapping. In addition, Fig. 15(e) suggests that changing the bit matching order between two multiplication operands can further improve the accuracy. This result is significant as it demonstrates that one needs to apply both optimizations in order to maximize the accuracy. In other words, a random ordering between LFSR bits and swappers cannot be a substitute for out-of-order bit matching between multiplication operands.

E. Design Exploration for All Input Values

1) *Design of Random Source:* So far we have assumed a fixed input value, which is 0.6. In practice, an SNG is designed once but used for any input value. A random source designed for one input value may not perform well for other input values. To see the effect of different input values, we first fix our SNG parameters (see below), and then evaluate the SNG for all different input values.

The main design issue in our SNG is the design of random source, which includes: 1) LFSR taps; 2) the ordering between LFSR bits and swappers; and 3) the bit matching order between multiplication operands. We use the weight ($=b$) of 1, the group size ($=v$) of 32. We consider a circuit with two SNGs and one SC multiplication. For evaluation purpose we convert the output bitstream into conventional BN, and see how closely the output value matches the target multiplication result.

The quality of the random source in our SNG depends on the input to the intragroup randomization circuit shown in

Fig. 9, namely, A_4, A_3, A_2, A_1, A_0 values. To design the random source that would work well for many input values, we choose two input BNs, $X_1 = 0.34275$ and $X_2 = 0.6875$, which generate $(0,1,0,1,0)$ and $(1,0,1,0,1)$, respectively, as input to the intragroup randomization circuit as $(A_4, A_3, A_2, A_1, A_0)$. We then use all four combinations of $X_1 \cdot X_1$, $X_1 \cdot X_2$, $X_2 \cdot X_1$, and $X_2 \cdot X_2$ in testing various random source designs. The goal is to find a random source design that works better than software random function (`rand()`) for those four input combinations. The following is one random source design found.

- 1) *LFSR Taps:* [31, 28, 26, 15, 8, 3].
- 2) *LFSR-Swapper Matching Order for SNG #1:* [9, 19, 8, 23, 11, 22, 26, 0, 3, 2, 4, 6, 31, 18, 14, 27, 24, 1, 5, 30, 17, 13, 15, 29, 28, 16, 7, 25, 12, 20, 10, 21].
- 3) *LFSR-Swapper Matching Order for SNG #2:* [7, 12, 23, 4, 19, 15, 21, 11, 6, 18, 17, 3, 28, 14, 16, 0, 13, 2, 27, 24, 20, 30, 8, 9, 25, 31, 1, 26, 22, 29, 5, 10].
- 4) *Bit Matching Order Between Operands:* [17, 28, 27, 2, 26, 9, 4, 30, 10, 20, 24, 15, 31, 14, 13, 1, 11, 16, 12, 5, 23, 8, 6, 18, 3, 22, 21, 29, 7, 25, 19, 0].

The above design is used in the following experiments. Note that the shuffling of bits takes almost no additional hardware as it is fixed at design time.

2) *Evaluation for Different Inputs:* Fig. 16 shows the evaluation result. We use 2^5 different input values for each multiplication operand, resulting in 2^{10} input combinations. The graph shows the mean, STD, and their ratio ($=\text{mean}/\text{STD}$), which can be seen as SNR (signal-to-noise ratio), comparing the following cases: conventional SNG (Conv.), the proposed SNG with optimized random sources specified above (Prop. Optimized), and the proposed SNG with software random function as the random source (Prop.+SW).

Fig. 16(a) shows that all three cases have zero bias in terms of the SC multiplication result. Thus lower STD means higher accuracy. Fig. 16(b) and (c) compares STD value for 2^5 -bit and 2^8 -bit SC bitstreams, respectively. In both cases, the proposed SNG produces lower STD, thus more accurate result, even compared with the Prop.+SW case. Compared with the conventional SNG, the proposed SNG is much more accurate by a large margin. All three schemes show lower STD when using longer bitstream. When one of the input values is close to 1, the proposed SNG gives STD values that are close to zero. This is because the proposed SNG can generate

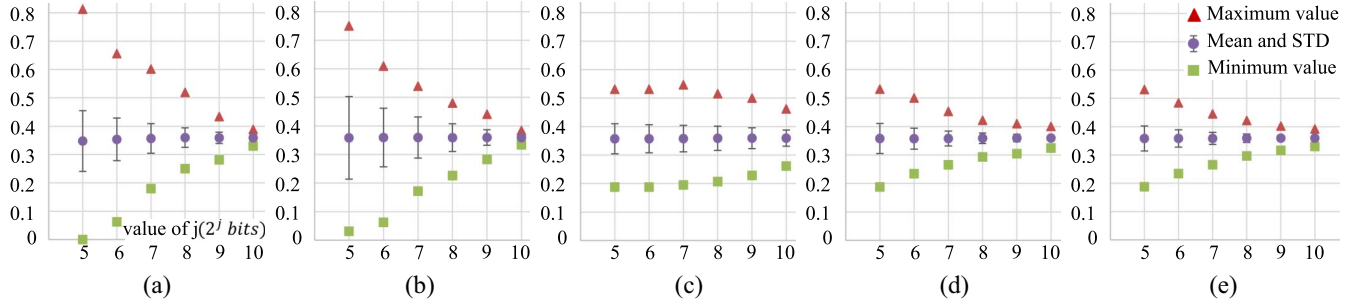


Fig. 17. SC multiplication accuracy, for different SNGs and stochastic bitstream lengths (input values: 0.6). (a) Conventional. (b) Shared [11]. (c) Prop.+Fibonacci. (d) Prop.+SW. (e) Prop. Optimized.

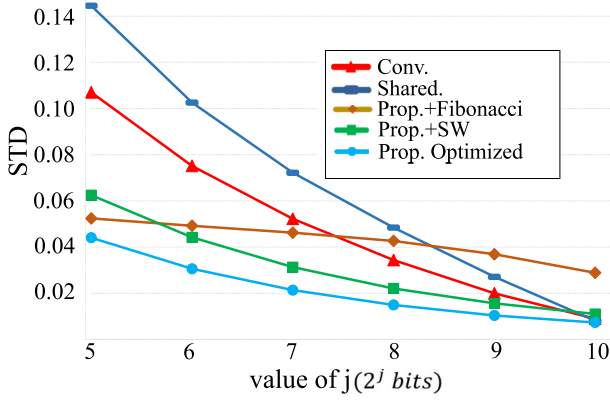


Fig. 18. STD comparison (input values: 0.6).

stochastic bitstreams whose mean is very close to the target value as shown in Fig. 13(b), whereas the conventional SNG does not.

Fig. 16(d) and (e) compares SNR for different bitstream lengths. We exclude from the graph the results where one of the inputs is greater than 0.97 since in such a case the SNR value takes extremely large values. Excluding those cases, the proposed SNG shows about $3\times$ better SNR on average over the conventional SNG for either bitstream length.

F. SC Multiplication Accuracy Comparison

Fig. 17 shows the SC multiplication results for different SNGs. To make comparison easier, we also show the STD value in a separate graph in Fig. 18. The shared case, which uses the conventional SNG with the LFSR shared between two operands, has the worst accuracy in terms of STD though the min/max values are similar to the conventional SNG case. On the other hand, the proposed SNG with optimized random source, which is specified in Section IV-E1, gives the most accurate result. It is even better than using the software random function with the proposed SNG. Lastly when using the Fibonacci LFSR with the proposed SNG, the accuracy is low especially for long bitstreams. These results confirm that the proposed SNG with optimized random source can generate very high quality stochastic bitstreams, far exceeding that of the conventional SNG.

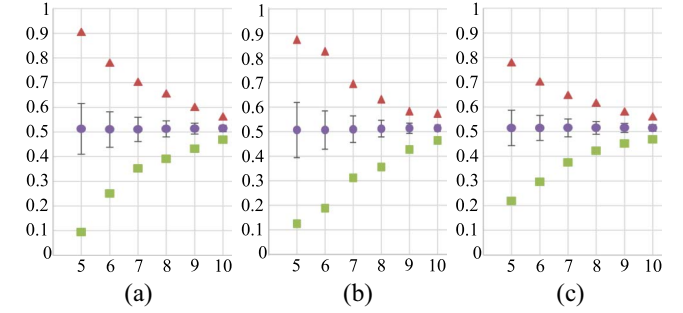


Fig. 19. Accuracy of scaled SC addition using MUX. (a) Conventional. (b) Shared [11]. (c) Prop. Optimized.

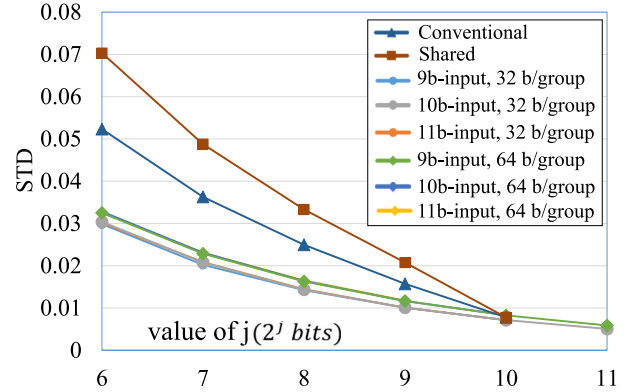


Fig. 20. Comparison of SC multiplication accuracy for different design parameters of our proposed SNG, along with the conventional and shared SNGs. Input value of 0.5 is used (for both operands).

G. SC Addition Accuracy Comparison

SC addition using MUX is less susceptible to correlation than SC multiplication. The reason is that the MUX used in SC addition selects one input while ignoring the other depending on the select signal. To confirm this we have conducted SC addition using operand values of 0.217 and 0.812 over 10000 iterations. The result which is shown in Fig. 19 demonstrates that our SNG indeed works well with SC addition. Our proposed SNG shows similar accuracy as the conventional SNG when full bitwidth is used, and shows higher accuracy when PP is used.

H. Proposed SNG With Different Design Parameters

Fig. 20 compares multiplication accuracy for different designs of the proposed SNG, along with that of the

conventional and shared SNGs. As SC multiplication gives the worst accuracy when inputs are 0.5, we compare the accuracy of SNGs using this value. We evaluate the effect of different input lengths, using 9 and 11 bits in addition to 10. As shown in Fig. 20 changing the input length in the proposed SNG hardly changes the result. We also compare the effect of different group sizes in our scheme, using 16 in addition to 32. The SNG with 16 groups produces slightly worse result than the SNG with 32 groups, but compared with the conventional and shared SNGs it still shows much better result. The 32-bit PP accuracy is not evaluated, since the SNG with 16 groups has 64 bits in each group.

V. CONCLUSION

One of the biggest problems with SC is the high overhead of generating stochastic bitstreams. To address the problem we presented a new SNG that is both more efficient and more accurate than the conventional SNG based on an LFSR and a comparator. The proposed SNG achieves high efficiency by generating multiple bits simultaneously through bit shuffling. To improve accuracy we have optimized random source design. Our experimental results demonstrate that our proposed SNG outperforms the existing approaches in terms of area, power, energy, and multiplication accuracy.

REFERENCES

- [1] K. Kim, J. Lee, and K. Choi, "An energy-efficient random number generator for stochastic circuits," in *Proc. 21st Asia South Pac. Design Autom. Conf. (ASP-DAC)*, Jan. 2016, pp. 256–261.
- [2] B. R. Gaines, "Stochastic computing systems," in *Advances in Information Systems Science*. Boston, MA, USA: Springer, 1969, ch. 2, pp. 37–172.
- [3] A. Alaghi and J. P. Hayes, "Exploiting correlation in stochastic circuit design," in *Proc. IEEE Int. Conf. Comput. Design (ICCD)*, Asheville, NC, USA, Oct. 2013, pp. 39–46.
- [4] B. D. Brown and H. C. Card, "Stochastic neural computation. I. Computational elements," *IEEE Trans. Comput.*, vol. 50, no. 9, pp. 891–905, Sep. 2002.
- [5] V. C. Gaudet and A. C. Rapley, "Iterative decoding using stochastic computation," *Electron. Lett.*, vol. 39, no. 3, pp. 299–301, Feb. 2003.
- [6] P. Li *et al.*, "Case studies of logical computation on stochastic bit streams," in *Integrated Circuit and System Design. Power and Timing Modeling, Optimization and Simulation*. Heidelberg, Germany: Springer, 2013, pp. 235–244.
- [7] A. Alaghi, C. Li, and J. P. Hayes, "Stochastic circuits for real-time image-processing applications," in *Proc. DAC*, Austin, TX, USA, 2013, pp. 1–6.
- [8] V. K. Chippa, S. T. Chakradhar, K. Roy, and A. Raghunathan, "Analysis and characterization of inherent application resilience for approximate computing," in *Proc. DAC*, Austin, TX, USA, 2013, pp. 1–9.
- [9] W. Qian, X. Li, M. D. Riedel, K. Bazargan, and D. J. Lilja, "An architecture for fault-tolerant computation with stochastic logic," *IEEE Trans. Comput.*, vol. 60, no. 1, pp. 93–105, Jan. 2011.
- [10] V. K. Chippa, S. Venkataramani, K. Roy, and A. Raghunathan, "StoRM: A stochastic recognition and mining processor," in *Proc. ISLPED*, La Jolla, CA, USA, 2014, pp. 39–44.
- [11] H. Ichihara, S. Ishii, D. Sunamori, T. Iwagaki, and T. Inoue, "Compact and accurate stochastic circuits with shared random number sources," in *Proc. IEEE Int. Conf. Comput. Design (ICCD)*, Seoul, South Korea, Oct. 2014, pp. 361–366.
- [12] A. Singhee and R. A. Rutenbar, *Novel Algorithms for Fast Statistical Analysis of Scaled Circuits*, vol. 46. Amsterdam, The Netherlands, Springer, 2009.
- [13] J. Hartmann and G. Kemnitz, "How to do weighted random testing for bist?" in *Proc. ICCAD*, Santa Clara, CA, USA, 1993, pp. 568–571.
- [14] F. Muradali, V. K. Agarwal, and B. Nadeau-Dostie, "A new procedure for weighted random built-in self-test," in *Proc. ITC*, Washington, DC, USA, 1990, pp. 660–669.
- [15] A. Alaghi and J. P. Hayes, "Fast and accurate computation using stochastic circuits," in *Proc. DATE*, Dresden, Germany, 2014, pp. 1–4.
- [16] K. Kim *et al.*, "Dynamic energy-accuracy trade-off using stochastic computing in deep neural networks," in *Proc. DAC*, Austin, TX, USA, 2016, pp. 1–6.
- [17] P. K. Gupta and R. Kumaresan, "Binary multiplication with PN sequences," *IEEE Trans. Acoust. Speech Signal Process.*, vol. ASSP-36, no. 4, pp. 603–606, Apr. 1988.



Aidyn Zhakatayev received the B.S. degree in electrical engineering and computer science from the Ulsan National Institute of Science and Technology, Ulsan, South Korea, in 2016, where he is currently pursuing the M.S. degree in computer science and engineering.

His current research interests include acceleration of deep neural networks, reconfigurable architectures, and stochastic computing.



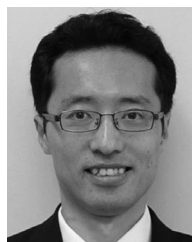
Kyounghoon Kim received the B.S. and M.S. degrees in computer science engineering from Seoul National University, Seoul, South Korea, in 2003 and 2005, respectively, and the Ph.D. degree from the Department of Electrical and Computer Engineering, Seoul National University, in 2016.

He has been with the DMC Research and Development Center, Samsung Electronics, Seoul, since 2005. He was a Researcher with GIS Soft, Bangkok, Thailand, from 1998 to 2003. His current research interests include deep learning and pattern recognition systems utilizing hardware acceleration design with parallel processing.



Kiyoung Choi (M'88–SM'08–F'17) received the B.S. degree in electronics engineering from Seoul National University, Seoul, South Korea, in 1978, the M.S. degree in electrical and electronics engineering from the Korea Advanced Institute of Science and Technology, Seoul, in 1980, and the Ph.D. degree in electrical engineering from Stanford University, Stanford, CA, USA, in 1989.

From 1989 to 1991, he was with Cadence Design Systems, Inc., San Jose, CA, USA. In 1991, he joined the faculty of the Department of Electrical and Computer Engineering, Seoul National University. His current research interests include computer-aided electronic systems design, low-power systems design, computer architecture, and machine learning.



Jongeun Lee (M'01) received the B.S. and M.S. degrees in electrical engineering and the Ph.D. degree in electrical engineering and computer science from Seoul National University, Seoul, South Korea, in 1997, 1999, and 2004, respectively.

Since 2009, he has been with the faculty of School of Electrical and Computer Engineering, Ulsan National Institute of Science and Technology, Ulsan, South Korea. His current research interests include domain-specific processors, reconfigurable architectures, and compilers.